IB DP COMPUTER SCIENCE

Complete Cheat Sheet

Full Syllabus | HL + SL | Score a 7

◆ Topics Covered	◆ Format
System Fundamentals	20 Pages, A4 Printable
Computer Organization	Exam Tips + Worked Examples
Networks & Security	Pseudocode Reference
Algorithms & Programming	Visual Diagrams
Abstract Data Structures	HL Extensions Labeled
Resource Management	Night-Before Checklist

Designed for students targeting a 6–7

How to Use This Cheat Sheet

This document is your exam-ready reference. Follow this approach:

- Read the section header — locate your topic quickly using the colour coding
- Use code blocks to memorize pseudocode syntax — IB is very strict on this
- Check Exam Tip boxes before your exam — these are high-yield insights
- Study Warning boxes to avoid common mark-losing mistakes
- Use the final checklist the night before — covers everything examiners test

Syllabus Roadmap

#	Section	SL / HL
1	System Fundamentals	SL + HL
2	Computer Organization	SL + HL
3	Networks	SL + HL
4	Computational Thinking & Programming	SL + HL
5	Abstract Data Structures	SL + HL
6	Resource Management	SL + HL
7	HL Extensions (OOP, Recursion, Advanced Algorithms)	HL only

Colour Guide

- Blue cards = Core concepts
- Teal / green cards = HL content (clearly labeled)
- Yellow cards = Common mistakes / warnings
- Code blocks = Pseudocode / algorithms

Section 1 — System Fundamentals

1.1 Data Types

Data Type	Description & Examples
Integer	Whole numbers: 0, 42, -7
Real / Float	Decimal numbers: 3.14, -0.5
Boolean	TRUE or FALSE only
Character	Single symbol: 'A', '3', '@'
String	Sequence of characters: "Hello"
Array	Fixed-size ordered collection of same type

1.2 Number Systems

System	Base / Digits
Binary	Base 2 — digits: 0, 1
Denary (Decimal)	Base 10 — digits: 0–9
Hexadecimal	Base 16 — digits: 0–9, A–F

Binary ↔ Denary Conversion

```
Denary 45 → Binary:
  45 ÷ 2 = 22 r1
  22 ÷ 2 = 11 r0
  11 ÷ 2 =  5 r1
   5 ÷ 2 =  2 r1
   2 ÷ 2 =  1 r0
   1 ÷ 2 =  0 r1   → Read remainders bottom to top: 101101

Binary 101101 → Denary:
  32 + 0 + 8 + 4 + 0 + 1 = 45
```

Hexadecimal ↔ Binary (group into nibbles)

```
Binary:  1010 1111  →  Hex: AF
  A = 1010 (10), F = 1111 (15)

Hex 3B → Binary: 0011 1011 → Denary: 59
```

1.3 Logic Gates

Gate	Symbol / Operation
AND	Output = 1 only if BOTH inputs are 1
OR	Output = 1 if AT LEAST ONE input is 1
NOT	Output = inverted input (1→0, 0→1)
NAND	Output = NOT(AND) — universal gate
NOR	Output = NOT(OR)
XOR	Output = 1 if inputs are DIFFERENT

★ Logic Gate Exam Tips

- Always fill in complete truth tables — IB awards 1 mark per correct row
- NAND and NOR are universal — any circuit can be built from them alone
- XOR: output is 1 when inputs DIFFER — used in half adders
- Show all working when simplifying Boolean expressions

1.4 Hardware vs Software

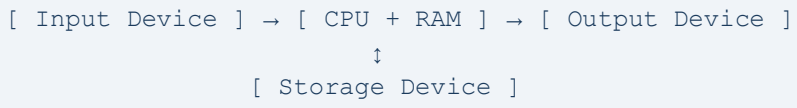
Hardware	Software
Physical components you can touch	Programs and data — intangible
CPU, RAM, HDD, keyboard, screen	OS, applications, compilers
Fixed functionality (without reprogramming)	Easily modified or replaced
Input, Process, Output, Storage	System software vs Application software

1.5 Input / Output / Storage Devices

Input Devices	Output Devices	Storage
Keyboard, mouse	Monitor, printer	HDD, SSD (secondary)
Microphone, scanner	Speakers, projector	RAM (primary — volatile)
Webcam, touchscreen	Haptic feedback	ROM (non-volatile)
Barcode reader, GPS	Braille display	USB flash, optical disc

1.6 System Diagrams

Basic Computer System



1.7 Operating System Functions

Function	Description
Process Management	Schedules & runs programs, allocates CPU time
Memory Management	Allocates RAM, handles virtual memory
File Management	Creates, reads, writes, deletes files
Device Management	Controls hardware via drivers
Security / User Interface	Login, permissions, GUI or CLI

⚠ Common Mistakes — System Fundamentals

- Confusing RAM (volatile, temporary) with ROM (non-volatile, permanent)
- Forgetting that hardware includes peripherals, not just the CPU
- Binary conversion: read remainders bottom-to-top, not top-to-bottom
- XOR is NOT the same as OR — always check when inputs are identical

Section 2 — Computer Organization

2.1 CPU Components

Component	Function
ALU (Arithmetic Logic Unit)	Performs arithmetic (+−×÷) and logical (AND/OR/NOT) operations
CU (Control Unit)	Directs data flow, coordinates CPU operations, interprets instructions
Registers	Ultra-fast temporary storage inside CPU (PC, MAR, MDR, Accumulator)
Cache	High-speed memory between CPU and RAM — reduces latency (L1, L2, L3)
Clock	Generates pulses to synchronise operations — measured in GHz
Buses	Data bus (data), Address bus (location), Control bus (signals)

2.2 Fetch–Decode–Execute Cycle

FDE Cycle Steps

```
FETCH:   PC → MAR; Memory[MAR] → MDR; MDR → IR; PC++
DECODE:  CU interprets instruction in IR
EXECUTE: ALU performs operation; result → Accumulator or Memory
```

Key Registers

Register	Purpose
PC (Program Counter)	Holds address of next instruction
MAR (Memory Address Register)	Holds address currently being accessed
MDR (Memory Data Register)	Holds data read from / written to memory
IR (Instruction Register)	Holds current instruction being decoded
Accumulator	Stores intermediate results of ALU operations

2.3 Memory Types

Memory Type	Volatile?	Speed / Use
Registers	Yes	Fastest — inside CPU
Cache (L1/L2/L3)	Yes	Very fast — between CPU and RAM
RAM	Yes	Fast — running programs and data
ROM	No	Slow — stores BIOS/boot firmware
Secondary (HDD/SSD)	No	Slowest — permanent storage

★ FDE Cycle Exam Tips

- Know exact register names: MAR, MDR, IR, PC, Accumulator
- PC increments DURING fetch (before execute)
- Distinguish between cache hit (fast) and cache miss (must fetch from RAM)
- Clock speed affects FDE rate — more GHz = more cycles per second

Section 3 — Networks

3.1 Network Types

Type	Description
PAN (Personal Area Network)	Very short range — Bluetooth, < 10m
LAN (Local Area Network)	Single building/campus — Ethernet, WiFi
MAN (Metropolitan Area Network)	City-wide — ISP infrastructure
WAN (Wide Area Network)	Country/global — internet, leased lines
VLAN (Virtual LAN)	Logical segments on physical LAN — improves security

3.2 Network Topologies

Topology	Advantages	Disadvantages
Bus	Simple, cheap	Single cable failure breaks all
Star	Easy to add nodes, fault isolated	Depends on central switch/hub
Ring	Equal access, predictable	One failure breaks the ring
Mesh	Highly redundant, fault-tolerant	Expensive, complex cabling

3.3 Internet Structure

> `Data travels: Application → Transport → Network → Data Link → Physical`
> *This is the TCP/IP model (OSI has 7 layers; IB uses 4-layer TCP/IP)*

Component	Function
Router	Routes packets between networks using IP addresses
Switch	Connects devices on same LAN using MAC addresses
Hub	Broadcasts all data to all devices (obsolete)
Modem	Modulates/demodulates signals for ISP connection
DNS Server	Resolves domain names to IP addresses
DHCP Server	Assigns IP addresses automatically to devices
Firewall	Filters traffic by rules — blocks unauthorised access

★ Networks Exam Tips

- Router uses IP addresses (Layer 3); Switch uses MAC addresses (Layer 2)
- DNS = phonebook of the internet — name to IP address
- Packet switching: data split into packets, each takes independent route
- Know the difference between LAN and WAN — IB often asks for 2 differences

3.4 Protocols

Protocol	Purpose
HTTP / HTTPS	Web browsing — HTTPS adds TLS encryption
FTP	File transfer between computers
SMTP / POP3 / IMAP	Email — SMTP sends, POP3/IMAP retrieves
TCP	Reliable, ordered delivery — checks for errors, resends
UDP	Fast, unreliable delivery — streaming, VoIP, gaming
IP	Addressing and routing of packets across networks
SSH	Secure remote terminal access (encrypted)
DNS	Resolves hostnames to IP addresses

3.5 Network Security

Threat / Attack	Description
Malware	Malicious software: virus, worm, trojan, ransomware
Phishing	Fake emails/sites to steal credentials
DDoS	Overwhelm server with traffic from many sources
Man-in-the-Middle	Attacker intercepts communication between two parties
SQL Injection	Inserting SQL code into input fields to access DB
Brute Force	Systematically trying all password combinations

Security Countermeasures

Countermeasure	How It Helps
Encryption (TLS/SSL)	Scrambles data so only intended recipient can read it
Firewall	Blocks unauthorised inbound/outbound traffic
Two-Factor Auth	Requires second verification — reduces stolen credential risk
Antivirus Software	Detects and removes known malware signatures
Strong Password Policy	Enforces length, complexity, expiry to resist brute force
VPN	Encrypts tunnel between device and server — hides data from ISP

⚠ Common Mistakes — Networks

- Confusing encryption (scrambles data) with hashing (one-way, for passwords)
- TCP and UDP are NOT interchangeable: TCP = reliable, UDP = fast
- A firewall does NOT prevent phishing — user education is needed
- WPA2/WPA3 (WiFi encryption) is different from HTTPS (web encryption)

Section 4 — Computational Thinking & Programming

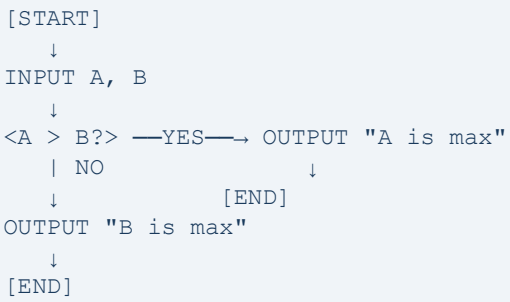
4.1 Algorithmic Thinking

Concept	Meaning
Decomposition	Breaking a large problem into smaller sub-problems
Pattern Recognition	Identifying similarities to apply known solutions
Abstraction	Removing unnecessary detail, focusing on essentials
Algorithm Design	Step-by-step instructions to solve a problem

4.2 Flowchart Symbols

Symbol Shape	Meaning
Oval / Rounded rectangle	START / STOP (Terminator)
Rectangle	Process (assignment, calculation)
Parallelogram	Input / Output
Diamond	Decision (Yes/No or True/False)
Arrow	Flow of control

Flowchart: Find Maximum of Two Numbers



★ Algorithm Exam Tips

- Flowchart decisions must have exactly TWO exit paths (Yes and No)
- Always label arrows on decision diamonds
- Trace tables: go step-by-step, show every variable change
- Abstraction = hiding complexity, not deleting detail

4.3 IB Pseudocode Reference (CRITICAL)

IB uses its own pseudocode style — memorise these keywords exactly

Variables & Assignment

```
X = 5               // Assign value
NAME = "Alice"      // String assignment
TOTAL = TOTAL + 1   // Update variable
```

Input / Output

```
INPUT X             // Read from user
OUTPUT X            // Print to screen
OUTPUT "Result: ", X
```

Conditional — IF / ELSE

```
IF X > 0 THEN
    OUTPUT "Positive"
ELSE IF X < 0 THEN
    OUTPUT "Negative"
ELSE
    OUTPUT "Zero"
END IF
```

Loops — LOOP / WHILE / FOR

```
// Count-controlled loop
loop COUNT from 1 to 10
    OUTPUT COUNT
end loop

// Condition-controlled loop
WHILE X > 0 DO
    X = X - 1
END WHILE

// DO...WHILE (executes at least once)
DO
    INPUT GUESS
WHILE GUESS <> ANSWER
```

Procedures & Functions

```
// Procedure (no return value)
PROCEDURE GREET(NAME)
    OUTPUT "Hello ", NAME
END PROCEDURE

// Function (returns a value)
FUNCTION SQUARE(N)
    RETURN N * N
END FUNCTION

// Calling them:
GREET("Alice")
RESULT = SQUARE(5)   // RESULT = 25
```

⚠ Pseudocode Mistakes

- IB uses END IF, END WHILE, END LOOP — not just closing brackets
- <> means NOT EQUAL in IB pseudocode (not !=)
- Use RETURN in functions — procedures do NOT return values
- Indentation matters for readability — examiners expect it

4.4 Searching Algorithms

Linear Search

```
FUNCTION LINEAR_SEARCH(ARR, TARGET)
    loop I from 0 to len(ARR)-1
        IF ARR[I] = TARGET THEN
            RETURN I
        END IF
    end loop
    RETURN -1   // Not found
END FUNCTION
// Time: O(n)  |  Works on unsorted arrays
```

Binary Search (array MUST be sorted)

```
FUNCTION BINARY_SEARCH(ARR, TARGET)
    LOW = 0
    HIGH = len(ARR) - 1
    WHILE LOW <= HIGH DO
        MID = (LOW + HIGH) DIV 2
        IF ARR[MID] = TARGET THEN
            RETURN MID
        ELSE IF ARR[MID] < TARGET THEN
            LOW = MID + 1
        ELSE
            HIGH = MID - 1
        END IF
    END WHILE
    RETURN -1
END FUNCTION
// Time: O(log n)  |  MUST be sorted first!
```

4.5 Sorting Algorithms

Bubble Sort

```
PROCEDURE BUBBLE_SORT(ARR)
    N = len(ARR)
    loop I from 0 to N-2
        loop J from 0 to N-2-I
            IF ARR[J] > ARR[J+1] THEN
                TEMP = ARR[J]
                ARR[J] = ARR[J+1]
                ARR[J+1] = TEMP
            END IF
        end loop
    end loop
END PROCEDURE
// Time: O(n²)  |  Simple but slow
```

Selection Sort

```
PROCEDURE SELECTION_SORT(ARR)
    N = len(ARR)
    loop I from 0 to N-2
        MIN_IDX = I
        loop J from I+1 to N-1
            IF ARR[J] < ARR[MIN_IDX] THEN
                MIN_IDX = J
            END IF
        end loop
        SWAP ARR[I] with ARR[MIN_IDX]
    end loop
END PROCEDURE
// Time: O(n²)  |  Fewer swaps than Bubble Sort
```

Algorithm	Best	Average	Worst	Stable?
Bubble Sort	O(n)	O(n²)	O(n²)	Yes
Selection Sort	O(n²)	O(n²)	O(n²)	No
Linear Search	O(1)	O(n)	O(n)	N/A
Binary Search	O(1)	O(log n)	O(log n)	N/A

★ Searching & Sorting Exam Tips

- Binary search requires sorted data — state this in every answer
- Trace bubble sort row-by-row: show array state after each pass
- Binary search reduces search space by HALF each step (log base 2)
- IB often asks: 'State one advantage of binary over linear search'

4.6 Algorithm Complexity

Big-O Notation measures how time/space scales with input size n

Notation	Name	Example
O(1)	Constant	Array access by index
O(log n)	Logarithmic	Binary search
O(n)	Linear	Linear search, array traversal
O(n log n)	Linearithmic	Merge sort, quicksort (avg)
O(n^2)	Quadratic	Bubble sort, selection sort
O(2^n)	Exponential	Brute-force subset problems

Worked Complexity Example

```
// What is the complexity of this code?
loop I from 1 to N           // runs N times
    loop J from 1 to N       // runs N times each
        OUTPUT I * J
    end loop
end loop

// Total operations: N × N = N²  → O(n²)
```

HL — Space Complexity & Recursion Depth

- Space complexity = memory used relative to n
- Recursive algorithms use O(n) stack space for n recursive calls
- Iterative solutions often have better space complexity O(1)
- Merge sort: O(n log n) time BUT O(n) space — needs extra array

⚡ Complexity Quick Reference

- Best: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$
- A single loop = O(n), nested loops = $O(n^2)$
- Halving the search space each step = O(log n)
- Always state: 'worst case complexity is...' in IB answers

Section 5 — Abstract Data Structures

5.1 Arrays

Property	Detail
Definition	Fixed-size, ordered, same data type
Indexing	Zero-based: ARR[0], ARR[1], ..., ARR[N-1]
Access Time	O(1) — direct index access
Insertion/Deletion	O(n) — must shift elements
Use Case	Lookup tables, storing exam scores, matrices

```
// Declare and access array
SCORES = [85, 72, 91, 67, 88]
OUTPUT SCORES[0]    // 85
SCORES[2] = 95      // Update index 2

// 2D Array (matrix)
GRID[ROW][COL]
GRID[0][0] = 1
```

5.2 Linked Lists

Structure: Each node contains DATA + POINTER to next node

```
[10|→] → [20|→] → [30|→] → [null]
 Head                       Tail
```

Operation	Linked List	Array
Access by index	O(n) — must traverse	O(1) — direct
Insert at head	O(1)	O(n) — shift required
Insert at tail	O(n)	O(1) if space available
Delete a node	O(n) to find, O(1) to remove	O(n) — shift required
Memory	Dynamic — grows as needed	Fixed at declaration

5.3 Stacks

LIFO — Last In, First Out

```
PUSH 10 → [10]   PUSH 20 → [10,20]   POP → returns 20, stack=[10]
```

Operation	Description
PUSH(item)	Add item to TOP of stack
POP()	Remove and return item from TOP
PEEK() / TOP()	Return TOP item without removing
isEmpty()	Returns TRUE if stack has no items
Stack Overflow	PUSH when stack is full

Stack Underflow	POP when stack is empty

Use Cases: Function call stack, undo operations, expression evaluation, browser back button

5.4 Queues

FIFO — First In, First Out

```
ENQUEUE 10,20,30 → [10,20,30]   DEQUEUE → returns 10, queue=[20,30]
```

Operation	Description
ENQUEUE(item)	Add item to REAR of queue
DEQUEUE()	Remove and return item from FRONT
isEmpty()	Returns TRUE if queue has no items
Queue Overflow	ENQUEUE when queue is full
Queue Underflow	DEQUEUE when queue is empty

Use Cases: Print queues, CPU scheduling, BFS graph traversal, buffering data streams

5.5 Data Structures Comparison

Structure	Key Property	Best Use Case
Array	Random access O(1)	Known size, frequent lookup
Linked List	Dynamic size	Frequent insert/delete
Stack (LIFO)	Last in, first out	Undo, recursion, DFS
Queue (FIFO)	First in, first out	Scheduling, BFS, buffers

Pseudocode: Stack Implementation

```
STACK = []
TOP = -1

PROCEDURE PUSH(ITEM)
    TOP = TOP + 1
    STACK[TOP] = ITEM
END PROCEDURE

FUNCTION POP()
    IF TOP = -1 THEN
        RETURN "UNDERFLOW"
    END IF
    ITEM = STACK[TOP]
    TOP = TOP - 1
    RETURN ITEM
END FUNCTION
```

⚠ Common Mistakes — Data Structures

- Stack uses TOP pointer; Queue uses FRONT and REAR pointers
- Array index starts at 0 — ARR[N] is OUT OF BOUNDS for size N
- LIFO vs FIFO: Stack = LIFO (last in, first out); Queue = FIFO
- Linked lists do NOT support O(1) random access — must traverse from head

Section 6 — Resource Management

6.1 Memory Management

Concept	Description
Heap	Dynamic memory allocated at runtime (malloc/new)
Stack Memory	Local variables, function call frames — auto-managed
Garbage Collection	Automatic reclaim of unused heap memory (Java, Python)
Memory Leak	Allocated memory never freed — program grows until crash
Virtual Memory	Uses disk space as extra RAM — slower, allows larger programs
Paging	Divides memory into fixed-size pages; maps virtual to physical
Segmentation	Divides memory into variable-size logical segments

6.2 Process Management

Process States Diagram

```
NEW → READY → RUNNING → TERMINATED
            ↕ (waiting for I/O)
          WAITING
```

Scheduling Algorithm	Description
FCFS (First Come First Served)	Queue order — simple but can cause long waits
Round Robin	Each process gets equal time slice — fair, responsive
Priority Scheduling	Highest priority runs first — can starve low-priority
Shortest Job First	Shortest burst time first — optimal average wait

6.3 Multitasking & Concurrency

- Concurrency: multiple tasks appear to run simultaneously (interleaved on one CPU)
- Parallelism: tasks truly run simultaneously on multiple CPU cores
- Race condition: two processes access shared data simultaneously — incorrect result
- Deadlock: two processes wait for each other's resources forever
- Mutex / Semaphore: synchronisation primitives to prevent race conditions

6.4 Secondary Storage Management

Concept	Description
File System	Organises files into directories/folders on disk
FAT / NTFS / ext4	Common file system formats (Windows/Linux)
Fragmentation	Files stored non-contiguously on disk — slows access

Defragmentation	Rearranges fragments into contiguous blocks
RAID	Redundant Array of Independent Disks — backup/speed

★ Resource Management Exam Tips

- Garbage collection prevents memory leaks — cite Java/Python as examples
- Virtual memory: slower than RAM, uses hard disk, allows larger programs
- Deadlock: requires 4 conditions — mutual exclusion, hold and wait, no preemption, circular wait
- RAID 1 = mirroring (backup); RAID 0 = striping (speed, no backup)

Section 7 — HL Extensions

HL Object-Oriented Programming (OOP)

OOP Concept	Definition
Class	Blueprint/template defining attributes and methods
Object	Instance of a class — a specific realisation
Encapsulation	Data hidden inside objects; accessed via methods only
Inheritance	Subclass inherits attributes/methods from superclass
Polymorphism	Same method name, different behaviour in different classes
Abstraction	Expose only necessary details; hide implementation
Constructor	Special method called when object is created — initialises attributes
Overriding	Subclass redefines a method from the superclass
Overloading	Same method name, different parameters in same class

Pseudocode: Class Definition

```
CLASS Animal
    PRIVATE name
    PRIVATE sound

    PUBLIC PROCEDURE Constructor(n, s)
        name = n
        sound = s
    END PROCEDURE

    PUBLIC PROCEDURE Speak()
        OUTPUT name, " says ", sound
    END PROCEDURE
END CLASS

CLASS Dog EXTENDS Animal
    PUBLIC PROCEDURE Fetch()
        OUTPUT name, " fetches the ball!"
    END PROCEDURE
END CLASS

myDog = new Dog("Rex", "Woof")
myDog.Speak()    // Rex says Woof
myDog.Fetch()    // Rex fetches the ball!
```

HL Recursion

A function that calls itself. Must have a BASE CASE (termination condition) and RECURSIVE CASE.

```
// Factorial: n! = n × (n-1)!
FUNCTION FACTORIAL(N)
    IF N = 0 OR N = 1 THEN    // BASE CASE
        RETURN 1
    ELSE                       // RECURSIVE CASE
        RETURN N * FACTORIAL(N - 1)
    END IF
```

```
END FUNCTION

// FACTORIAL(4) = 4 × FACTORIAL(3)
//              = 4 × 3 × FACTORIAL(2)
//              = 4 × 3 × 2 × FACTORIAL(1)
//              = 4 × 3 × 2 × 1 = 24
```

```
// Fibonacci sequence
FUNCTION FIB(N)
    IF N <= 1 THEN
        RETURN N              // Base cases: FIB(0)=0, FIB(1)=1
    ELSE
        RETURN FIB(N-1) + FIB(N-2)
    END IF
END FUNCTION

// WARNING: Exponential O(2^n) — not efficient for large N
```

⚠ Recursion Pitfalls

- Missing base case = infinite recursion = stack overflow
- Each recursive call adds a stack frame — deep recursion uses lots of memory
- Fibonacci is O(2^n) recursively — use iteration or memoization for efficiency
- In exams: always TRACE recursion step-by-step, show call stack

HL Advanced Algorithms

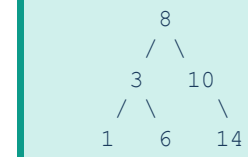
Merge Sort (HL)

```
// Divide and Conquer — O(n log n)
FUNCTION MERGE_SORT(ARR)
    IF len(ARR) <= 1 THEN
        RETURN ARR
    END IF
    MID = len(ARR) DIV 2
    LEFT  = MERGE_SORT(ARR[0..MID-1])
    RIGHT = MERGE_SORT(ARR[MID..end])
    RETURN MERGE(LEFT, RIGHT)
END FUNCTION

// Trace for [38, 27, 43, 3]:
// Split: [38,27] | [43,3]
// Split: [38]|[27] | [43]|[3]
// Merge: [27,38] | [3,43]
// Merge: [3,27,38,43] ✓
```

Binary Trees (HL)

Binary Search Tree (BST):



Left subtree < Root < Right subtree (always!)

Tree Traversals

Traversal	Order	Result on tree above
In-order	Left, Root, Right	1, 3, 6, 8, 10, 14 (sorted!)
Pre-order	Root, Left, Right	8, 3, 1, 6, 10, 14
Post-order	Left, Right, Root	1, 6, 3, 14, 10, 8

HL Object-Oriented Design Principles

Principle	Meaning in IB context
Single Responsibility	Each class has one purpose
Composition vs Inheritance	Prefer 'has-a' over 'is-a' where appropriate
Access Modifiers	PUBLIC = accessible anywhere; PRIVATE = class only
UML Class Diagram	Shows class name, attributes, methods, relationships

★ HL Exam Tips

- OOP: always define class attributes as PRIVATE — use getters/setters
- Inheritance: child class automatically has all parent methods/attributes
- Merge sort: explain Divide, Conquer, Combine — 3 phases
- In-order BST traversal gives SORTED output — worth memorising

Section 8 — Exam Mastery

17. Common Mistakes (Top 15)

1. Binary conversion direction
Read remainders BOTTOM to TOP, not top to bottom

2. Confusing RAM and ROM
RAM = volatile (lost on power off); ROM = non-volatile (permanent)

3. Binary Search on unsorted data
Always state: array MUST be sorted before binary search

4. Wrong IB pseudocode keywords
Use END IF, END WHILE, END LOOP — not endif or just }

5. Router vs Switch confusion
Router uses IP (between networks); Switch uses MAC (within LAN)

6. Treating TCP and UDP as identical
TCP = reliable, ordered, error-checked; UDP = fast, no guarantees

7. Stack overflow vs underflow
Overflow = push when full; Underflow = pop when empty

8. Array indexing off-by-one errors
Index 0 to N-1 — accessing ARR[N] is out of bounds

9. Missing base case in recursion
No base case = infinite recursion = stack overflow crash

10. Not explaining WHY in 'explain' questions
'Explain' needs cause AND effect — state + reason

11. Incomplete trace tables
Include ALL variables AND output column — every step

12. Confusing LIFO and FIFO
Stack = LIFO (last in first out); Queue = FIFO (first in first out)

13. Saying encryption and hashing are the same
Encryption is reversible (with key); hashing is one-way

14. Wrong CPU register for wrong step in FDE
Fetch uses MAR/MDR/IR/PC — not accumulator

15. Assuming higher clock speed always = faster computer
Also depends on: cache size, cores, RAM, bus speed, architecture

18. Exam Strategy

IB Command Terms — What They Mean

Command Term	What You Must Do
State / List	Give a fact — no explanation needed (1 mark each)
Define	Give the precise meaning of a term
Describe	Give key features/characteristics — factual account
Explain	State the fact AND give the reason — cause + effect
Analyse	Break down into components, examine each
Evaluate	Discuss strengths AND limitations — balanced judgement
Justify	Give reasons to support a conclusion
Construct	Create a diagram, table, or algorithm
Trace	Follow an algorithm step-by-step, showing all variable states

How to Approach Programming Questions

- Step 1: READ the entire question including any given code or data
- Step 2: IDENTIFY — what data structure? What algorithm pattern?
- Step 3: PLAN — write outline in pseudocode comments first
- Step 4: WRITE — use correct IB pseudocode syntax throughout
- Step 5: TRACE — test your algorithm with the given input values
- Step 6: CHECK — output matches expected? Edge cases handled?

Mark Allocation Strategy

Marks	Expected Response
1 mark	One correct fact or keyword
2 marks	One explained point OR two separate facts
4 marks	Two well-explained points with detail
6 marks	Balanced evaluation with advantages and limitations
Algorithm Q	Correct pseudocode + explanation of logic

★ Exam Day Tips

- Underline command terms first — they tell you exactly what to write
- Never leave a trace table blank — partial credit given for each correct row
- For 'evaluate' questions: write advantages first, then limitations, then conclusion
- Pseudocode: use consistent indentation — examiners reward clear structure
- Check mark allocation — 1-mark answers should be 1 sentence, not a paragraph

19. Key Pseudocode Patterns

Pattern 1: Sum of Array

```
TOTAL = 0
loop I from 0 to len(ARR)-1
    TOTAL = TOTAL + ARR[I]
end loop
OUTPUT TOTAL
```

Pattern 2: Find Maximum in Array

```
MAX = ARR[0]
loop I from 1 to len(ARR)-1
    IF ARR[I] > MAX THEN
        MAX = ARR[I]
    END IF
end loop
OUTPUT MAX
```

Pattern 3: Count Occurrences

```
COUNT = 0
loop I from 0 to len(ARR)-1
    IF ARR[I] = TARGET THEN
        COUNT = COUNT + 1
    END IF
end loop
OUTPUT COUNT
```

Pattern 4: Reverse an Array

```
LEFT = 0
RIGHT = len(ARR) - 1
WHILE LEFT < RIGHT DO
    TEMP = ARR[LEFT]
    ARR[LEFT] = ARR[RIGHT]
    ARR[RIGHT] = TEMP
    LEFT = LEFT + 1
    RIGHT = RIGHT - 1
END WHILE
```

Pattern 5: String Processing

```
// Count vowels in a string
VOWELS = "aeiouAEIOU"
COUNT = 0
loop I from 0 to len(STR)-1
    IF STR[I] is in VOWELS THEN
        COUNT = COUNT + 1
    END IF
end loop
```

Pattern 6: Nested Loop (2D Array)

```
// Process every element in a 2D grid
loop ROW from 0 to ROWS-1
    loop COL from 0 to COLS-1
        OUTPUT GRID[ROW][COL]
    end loop
end loop
```

⚡ Pseudocode Cheat Reference

- Assignment: X = value
- Not equal: X <> Y
- Integer division: A DIV B
- Modulo: A MOD B
- Logical: AND, OR, NOT
- String length: len(STR)

20. Final Revision — Night Before Exam

Night-Before Checklist

- System Fundamentals: binary/hex conversion, logic gates truth tables
- CPU: FDE cycle steps + register names (MAR, MDR, IR, PC, Accumulator)
- Networks: LAN/WAN, TCP/IP, router vs switch, common protocols
- Pseudocode: IF/ELSE/END IF, WHILE/END WHILE, loop/end loop, functions
- Searching: linear O(n) unsorted | binary O(log n) sorted-only
- Sorting: bubble $O(n^2)$, selection $O(n^2)$ — trace with given data
- Data Structures: stack LIFO, queue FIFO, array O(1) access, linked list O(n) access
- Resource Management: RAM vs ROM, virtual memory, garbage collection
- HL: OOP — class, object, encapsulation, inheritance, polymorphism
- HL: Recursion — always identify base case and recursive case

Quick Topic Bullets — Guaranteed Exam Topics

Topic	Key Fact	Typical Q Format
Binary	8 bits = 1 byte	Convert 45 to binary
Logic Gates	NAND is universal	Complete truth table
FDE Cycle	PC increments in Fetch	Describe role of MAR
TCP vs UDP	TCP reliable, UDP fast	State one difference
Binary Search	Must be sorted, O(log n)	Trace on given array
Stack	LIFO, push/pop	Write PUSH pseudocode
Recursion	Base + recursive case	Trace FACTORIAL(4)
OOP (HL)	Class = blueprint	Define encapsulation

Final Advice

- Pseudocode questions: write it — don't describe it
- Trace tables: show EVERY row, EVERY variable
- Describe vs Explain: explain needs the reason (because...)
- Use technical vocabulary: names, not descriptions
- For 6-mark evaluate: structure as advantages / limitations / conclusion

Good Luck — You've Got This! □